

SMART HAS

Inteligência de Dados com Oracle PL/SQL

AI Logistics Extension · React Native · Spring Boot · Angular · Oracle PL/SQL

INTEGRANTE



Leonardo Marsiarelli

RM: 555366

Turma: 3SIOA

Do MVP funcional à inteligência de dados no Oracle

1

Fases anteriores

Mobile (React Native), backend (Spring Boot + JWT + AI Logistics Extension) e dashboard (Angular) — MVP funcional e conectado.

2

Esta fase

Oracle PL/SQL: procedures e functions reutilizáveis direto no banco, integradas ao back-end Java.

3

Objetivo

Consolidar a interoperabilidade entre mobile, web, API REST e banco Oracle, com governança de dados e automação.

A camada Oracle como evolução do backend

1

NOVAS FUNCIONALIDADES

4 endpoints `/api/oracle/**`

Réplica automática de leituras críticas

Relatório de entregas por paciente

2

REFATORAÇÃO

Datasource Oracle isolado

Integração opcional (Optional<...>)

Falha nunca derruba o fluxo principal

3

VALOR AGREGADO

Regras reutilizáveis direto no banco

Mesma regra em Java e PL/SQL

Pronto para BI/relatórios externos

Modelo de dados: 4 tabelas, 1 domínio coeso

APP_USER

Pacientes e administradores

BLOOD_PRESSURE_READING

Leituras de sensor (pressão arterial)

MEDICATION_DELIVERY_REQUEST

AI Logistics Extension

SENSOR_ALERT

Alertas (novo nesta fase)

DER — relacionamentos

APP_USER (1) → BLOOD_PRESSURE_READING (N)

APP_USER (1) → MEDICATION_DELIVERY_REQUEST (N)

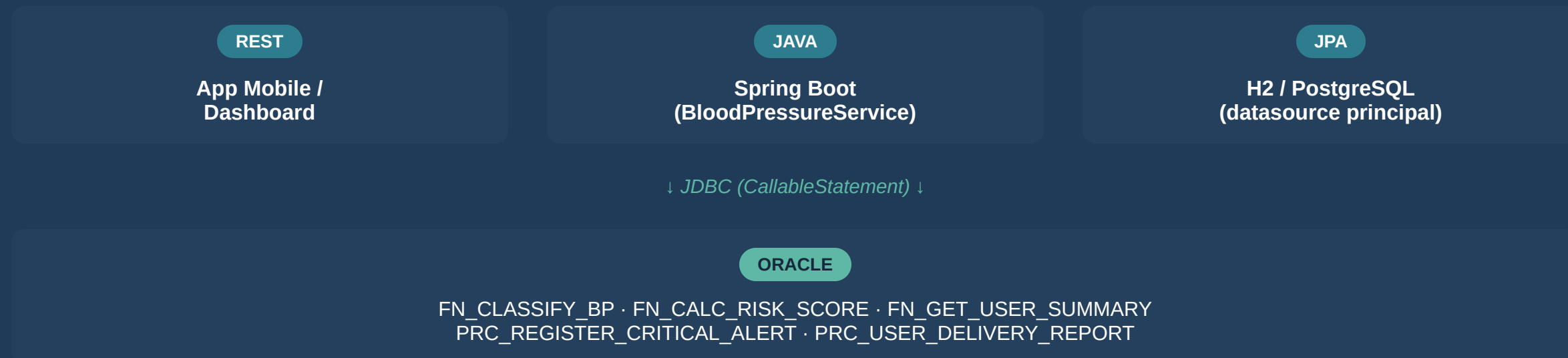
APP_USER (1) → SENSOR_ALERT (N)

BLOOD_PRESSURE_READING (1) → SENSOR_ALERT (N)

Diagrama completo em docs/DER_Oracle.png — validado contra Oracle XE 21c real.

Dois datasources, um backend só

H2/Postgres continua operacional (JPA); Oracle entra como camada adicional de inteligência — nunca substitui o fluxo principal.



Regras de negócio reutilizáveis no banco

Function	Assinatura	Propósito
FN_CLASSIFY_BP	<i>p_systolic, p_diastolic IN → VARCHAR2</i>	Classifica a pressão arterial (reutilizada pelas outras duas)
FN_CALC_RISK_SCORE	<i>p_systolic, p_diastolic IN → NUMBER</i>	Indicador de risco (0-100) da AI Logistics Extension
FN_GET_USER_SUMMARY	<i>p_user_id IN → VARCHAR2</i>	Dados formatados: resumo do paciente, via CURSOR

```
SELECT u.name, fn_classify_bp(r.systolic, r.diastolic) AS classificacao,
       fn_calc_risk_score(r.systolic, r.diastolic) AS risk_score
FROM blood_pressure_reading r JOIN app_user u ON u.id = r.user_id;
```

Automação de rotinas direto no banco

PRC_REGISTER_CRITICAL_ALERT

IF · EXCEPTION

Registra alerta quando a leitura é HAS_ESTAGIO_2/CRISE. Acionada pelo backend Java a cada nova leitura (REST → Java → JDBC → Oracle).

PRC_USER_DELIVERY_REPORT

CURSOR · LOOP · EXCEPTION · REF CURSOR

Total de pedidos/unidades via cursor explícito + detalhe via SYS_REFCURSOR, usado pela API.

REST (POST /api/readings) → Java (BloodPressureService) → JDBC (CallableStatement) → Oracle

Testado contra um Oracle XE 21c real

Não é só script revisado no papel — rodamos tudo contra um container Oracle real (gvenzl/oracle-xe:21-slim) via Docker.

Passo	Resultado
✓ Registro de paciente via API	Réplica automática criada em APP_USER (Oracle), casada por email
✓ POST /readings 195/128 (CRISE)	Leitura inserida + PRC_REGISTER_CRITICAL_ALERT → alerta CRITICO registrado
✓ POST /readings 115/75 (NORMAL)	Leitura replicada, nenhum alerta criado (confirma o IF da procedure)
✓ GET /oracle/users/{id}/summary	Resumo formatado retornado por FN_GET_USER_SUMMARY
✓ GET /oracle/.../delivery-report	Total + detalhe retornados por PRC_USER_DELIVERY_REPORT

Código, credenciais e vídeo

REPOSITÓRIO GITHUB

github.com/LeoMarsiarelli/Smart-HAS

/mobile · /backend · /web-admin · /database · /docs

VÍDEO DE DEMONSTRAÇÃO

[Inserir link do YouTube — não listado]

Demonstração prática do app + Oracle em funcionamento

CREDENCIAIS DE DEMONSTRAÇÃO

ADMIN

admin@smarthas.com

senha: admin123

PACIENTE

maria@smarthas.com

senha: paciente123

Conclusão

O Smart HAS consolidou a interoperabilidade entre mobile, web, API REST e banco Oracle: as mesmas regras de negócio agora vivem tanto em Java quanto em PL/SQL, reutilizáveis por qualquer cliente que converse com o banco.

1

Motor de regras → Machine Learning treinado

2

Oracle Autonomous Database em nuvem

3

Notificações push + testes E2E

SB

NG

RN

PL